Day 46: Learning-Rate Schedules (Step Decay, Cosine Annealing)

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 7, Day 5

Day 45 ended with an honest surprise: even with EXACT, deterministic full-batch gradients, a FIXED learning rate (lr=0.08) still caused real loss spikes late in training as the loss basin narrowed. Today builds two schedules from scratch (step decay, cosine annealing), confirms one of them calms that exact oscillation, and then makes a real, honest attempt at Day 44's still-unresolved question -- does a proper schedule finally get WideTwoBlockConvNetRGB past its 0.995 plateau? The answer, reached by actually running both schedules, is no -- and one attempt made things dramatically worse. Both results are reported in full, because a schedule fixing this network was never guaranteed, and this series has never hidden an inconvenient result.

0. Where Today Fits

Start here (no prior knowledge assumed): Every network since Day 37 has trained with ONE fixed learning rate the whole time. Day 45 found that even with perfectly exact, noise-free full-batch gradients, a fixed rate can still cause real loss spikes late in training, purely from overshooting a narrowing loss basin. Learning-rate schedules exist to fix exactly that: take big steps early when far from a minimum, small steps late when close to one. Today builds two schedules from scratch, confirms one calms Day 45's own oscillation, then uses both on Day 44's still-unfinished business -- a wide network that plateaued at train_acc=0.995 instead of 1.000.

1. Learning-Rate Schedules From Scratch

Start here (no prior knowledge assumed): A learning-rate schedule is just a function: give it the current epoch number, it gives you back the learning rate to use for that epoch. That's it -- no new network code, no new math for the forward/backward pass, just a different number fed into the SAME weight-update line that's been there since Day 37.

Topic specification: constant_schedule(lr0) always returns lr0. step_decay_schedule(lr0, decay_factor, decay_every) multiplies lr0 by decay_factor every decay_every epochs -- a staircase. cosine_annealing_schedule(lr0, lr_min, total_epochs) smoothly interpolates from lr0 down to lr_min following half a cosine wave.

Mathematics:

MATH

$$\text{step decay: } lr(epoch) = lr0 \times decay\_factor^{floor(epoch\ /\ decay\_every)}$$

$$\text{cosine annealing: } lr(epoch) = lr\_min + \tfrac{1}{2}(lr0 - lr\_min) \times (1 + \cos(\pi \times epoch\ /\ total\_epochs))$$

Implementation:

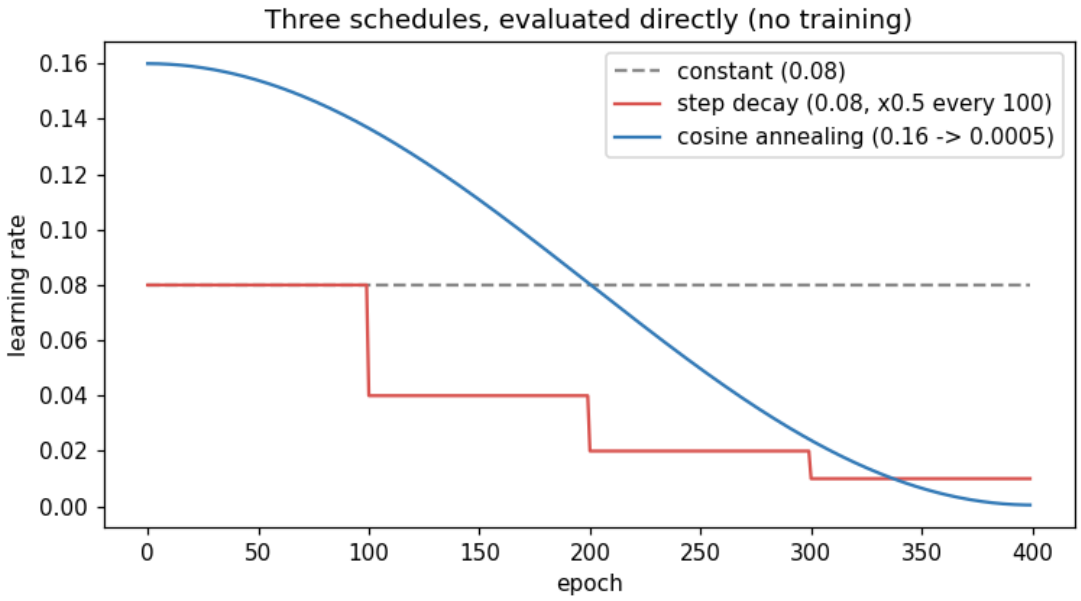
```
def step_decay_schedule(lr0, decay_factor, decay_every):
    def sched(epoch):
        return lr0 * (decay_factor ** (epoch // decay_every))
    return sched

def cosine_annealing_schedule(lr0, lr_min, total_epochs):
    def sched(epoch):
        return lr_min + 0.5 * (lr0 - lr_min) * (
            1 + np.cos(np.pi * epoch / total_epochs))
    return sched
```

Actual output:

```
Real, actually-evaluated lr values at several checkpoint epochs (no training
happening here -- these are the schedule functions called directly):
  epoch  constant(0.08)   step_decay    cosine
      0         0.08000      0.08000   0.16000
     50         0.08000      0.08000   0.15393
     99         0.08000      0.08000   0.13708
    100         0.08000      0.04000   0.13664
    150         0.08000      0.04000   0.11077
    199         0.08000      0.04000   0.08088
    200         0.08000      0.02000   0.08025
    250         0.08000      0.02000   0.04973
    299         0.08000      0.02000   0.02430
    300         0.08000      0.01000   0.02386
    350         0.08000      0.01000   0.00657
    399         0.08000      0.01000   0.00050
```

How it works, visually:



Three schedules, evaluated directly (no training) -- step decay's discrete jumps vs. cosine's smooth curve, both starting above the constant baseline.

Why choose this? Step decay is dead simple to reason about -- you always know exactly what lr will be at any epoch from one multiplication, and it needs only two numbers (decay_factor, decay_every) to tune.

Why NOT this (tradeoffs)? Cosine annealing needs no decay-schedule tuning beyond choosing lr_min and total_epochs, and its smooth curve avoids the sudden jumps that can visibly disturb training right at a decay boundary -- but it commits to a specific total_epochs up front, which step decay does not.

2. A Generic Scheduled Training Loop

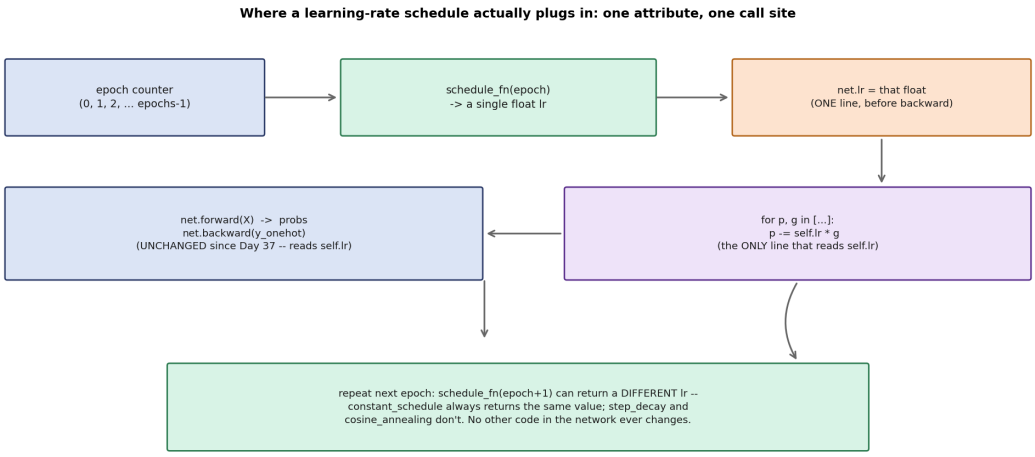
Start here (no prior knowledge assumed): Every network class since Day 37 already stores its learning rate as a plain mutable attribute (self.lr) and reads it in exactly one place: the weight update line inside backward(). That is the ENTIRE hook a schedule needs.

Topic specification: train_scheduled(net, X, y, epochs, schedule_fn) sets net.lr = schedule_fn(epoch) immediately before each backward() call. Passing schedule_fn = constant_schedule(lr) reproduces the old fixed-lr loop exactly -- a schedule is a strict generalization of every prior day's training loop, not a separate path.

Implementation:

```
def train_scheduled(net, X, y, epochs, schedule_fn, n_classes=4):
    y_onehot = one_hot(y, n_classes)
    losses, lrs = [], []
    for epoch in range(epochs):
        net.lr = schedule_fn(epoch)     # <-- the ONLY new line
        probs = net.forward(X)
        losses.append(cross_entropy_loss(probs, y_onehot))
        net.backward(y_onehot)
        lrs.append(net.lr)
    return losses, lrs
```

How it works, visually:



Where a schedule plugs in: one attribute (net.lr), one call site (backward()'s weight update). No other network code changes.

Why choose this? This design means every network class from Day 37 onward already supports schedules for free -- nothing about TwoBlockConvNetRGB or WideTwoBlockConvNetRGB needed to change.

Why NOT this (tradeoffs)? Because net.lr is just a plain attribute with no validation, a bug in a schedule_fn (e.g. returning a negative number, or forgetting to bound lr_min) would silently corrupt training with no error raised -- worth a sanity check when writing a new schedule.

3. Does a Schedule Calm Day 45's Late-Training Oscillation?

Start here (no prior knowledge assumed): Day 45's step-matched full-batch run (TwoBlockConvNetRGB, fixed lr=0.08, 350 epochs) found real loss spikes recurring from roughly epoch 75 through 320, with noise_std=0.264 over the second half -- purely from the fixed step size overshooting a narrowing loss basin. This topic reruns that EXACT setup (same dataset convention as Day 44/45: train = seed=42, n_per_class=50, all 200 images; test = a separate seed=999, n_per_class=20 generation, 80 images) with a step-decay schedule instead of the fixed rate.

Topic specification: step_decay_schedule(lr0=0.08, decay_factor=0.5, decay_every=100) -- lr halves every 100 epochs: 0.08 for epochs 0-99, 0.04 for 100-199, 0.02 for 200-299, 0.01 for 300-349.

Real-world scenario: This mirrors a real training pattern: a team notices their loss curve has sporadic spikes late in a run and, rather than re-tuning the base learning rate from scratch, adds a decay schedule on top of the rate that was already working reasonably well early on.

Actual output:

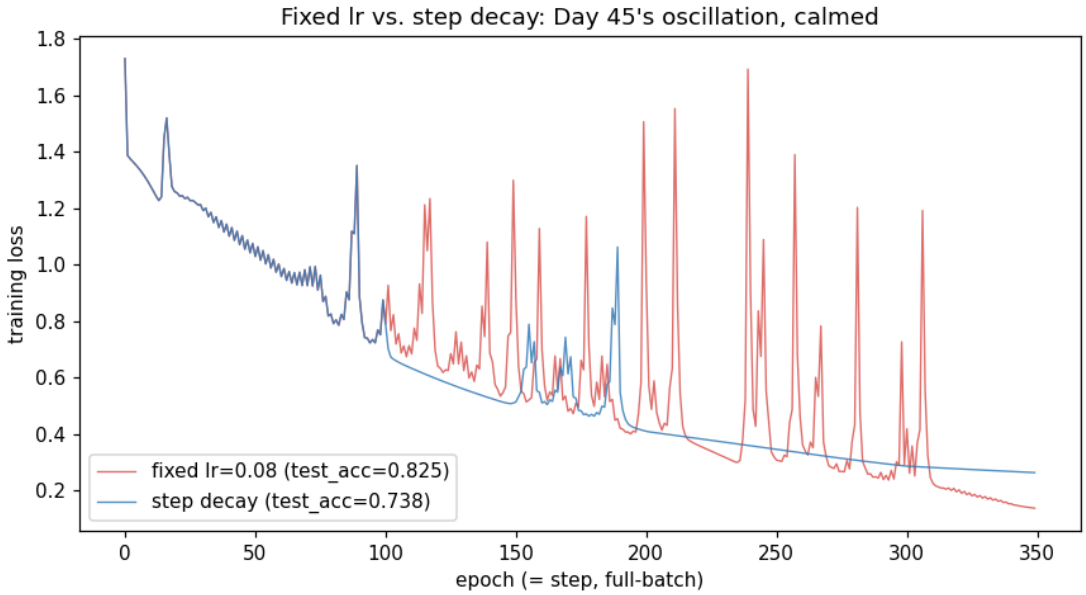
config	train_acc	test_acc	noise_std (2nd half)
Fixed lr=0.08	0.990	0.825	0.24795
Step decay 0.08→0.01	0.985	0.738	0.04935

Actual output from this lesson's run.

Actual output:

```
lr trajectory (step decay): epoch0=0.0800 epoch99=0.0800 epoch100=0.0400
epoch199=0.0400 epoch200=0.0200 epoch300=0.0100 epoch349=0.0100
noise_std over just the last 50 steps (step decay): 0.00002
```

How it works, visually:



Fixed lr (red) keeps spiking sharply through epoch ~320; step decay (blue) smooths out almost entirely once decay kicks in around epoch 100.

Line-by-line explanation

• Verified: step decay all but ELIMINATES the oscillation -- noise_std drops from 0.24795 to 0.04935 over the second half, and to just 0.00002 in the final 50 steps, where lr has decayed to 0.01.

• Honest complication: this did NOT come free. Both train and test accuracy dropped slightly (train_acc 0.990→0.985; test_acc 0.825→0.738). Day 45's oscillation, caused by the same fixed lr overshooting, may have been acting like an unintentional regularizer -- the noisy steps kept exploring slightly past the exact training optimum, which happened to generalize marginally better on this 80-image test set.

• This is a real, single-run correlation, not proof of a general 'noise helps' rule -- but it's exactly the kind of result worth flagging rather than smoothing over just because the noise metric moved the expected way.

Term refresher — regularization: Any technique that trades a little training-set fit for better generalization to unseen data. Explicit regularizers (L1/L2, dropout -- covered on later days) are added on purpose; here, a fixed learning rate's own instability may have acted as an ACCIDENTAL one.

Why choose this? Use a decaying schedule when you've actually OBSERVED oscillation or instability late in training, like Day 45's spikes -- it's a precise fix for that specific symptom.

Why NOT this (tradeoffs)? Don't reach for a schedule as a default 'better training' switch -- as Topic 5 shows, decaying a rate that was never unstable in the first place can make things slightly worse, not better.

4. Revisiting Day 44's Wide Network — Attempt 1: Cosine Annealing

Start here (no prior knowledge assumed): Day 44's WideTwoBlockConvNetRGB (f1=6, f2=12 -- double the filters of TwoBlockConvNetRGB) reached only train_acc=0.995 after 400 epochs at

fixed lr=0.08 -- every other network that day hit 1.000. A slower lr=0.05 was slower still, and a faster lr=0.12 was unstable (train_acc collapsed to 0.27 at epoch 100). This topic tries a cosine-annealing schedule that starts ABOVE the safe 0.08 rate, hoping the fast early steps help it converge fully before the schedule brings the rate back down.

Topic specification: cosine_annealing_schedule(lr0=0.16, lr_min=0.0005, total_epochs=400) -- 2x Day 44's safe fixed rate at epoch 0, decaying smoothly to near-zero by epoch 400. Dataset: Day 44's EXACT convention (train: seed=42, n_per_class=50, all 200 images; test: a SEPARATE seed=999, n_per_class=20 generation, 80 images) -- deliberately different from Day 45's own single-generation-plus-holdout convention used in Topic 3, and getting this wrong on the first attempt building today's lesson is exactly the kind of dataset-convention mismatch this series has flagged before.

Actual output:

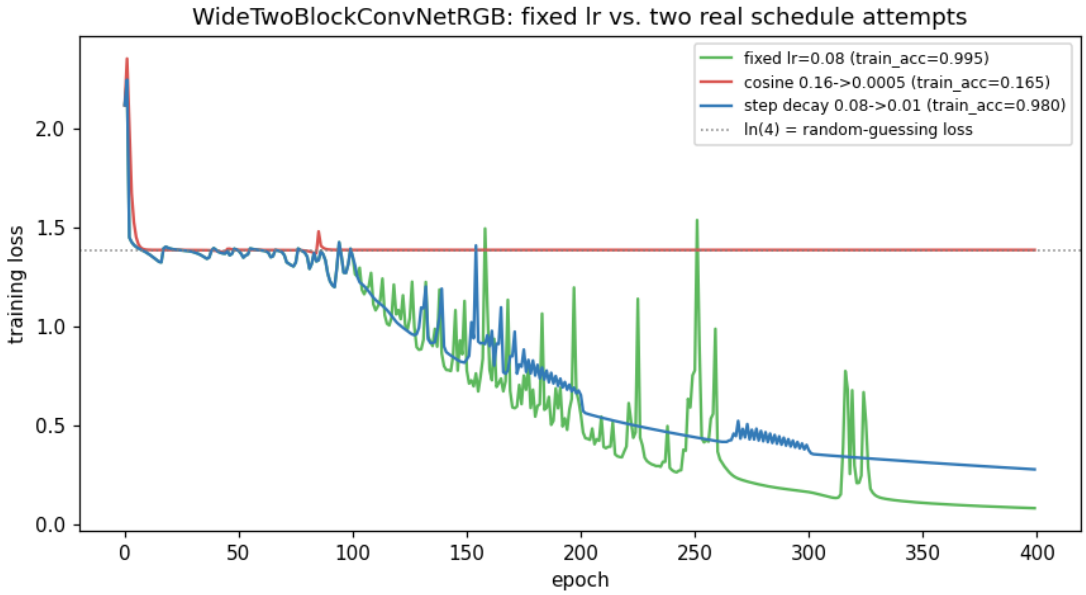
config	train_acc	test_acc
Fixed lr=0.08 (Day 44 baseline, reproduced)	0.995	0.537
Cosine 0.16→0.0005	0.165	0.225

Actual output from this lesson's run. Baseline matches Day 44's own documented numbers exactly (train_acc=0.995, test_acc=0.537).

Actual output:

```
loss trajectory: epoch0=2.1162 epoch99=1.3863 epoch199=1.3863 epoch299=1.3863
epoch399=1.3863  (ln(4)=1.3863 is pure random-guessing loss for 4 classes)
probs on 3 training images after 400 epochs:
[[0.25, 0.25, 0.25, 0.25], [0.25, 0.25, 0.25, 0.25], [0.25, 0.25, 0.25, 0.25]]
```

How it works, visually:



Cosine (red) collapses to exactly ln(4) by epoch ~15 and never recovers, even as its own schedule decays lr all the way to 0.0005 by epoch 400.

Line-by-line explanation

• Honest complication: cosine annealing starting at lr0=0.16 did not just fail to help -- it produced a WORSE, and apparently PERMANENT, collapse than Day 44's already-unstable fixed lr=0.12 (which reached train_acc=0.27, not 0.165).

• The network is predicting almost exactly uniform probability (0.25, 0.25, 0.25, 0.25) across all 4 classes for every image, and stays there for the remaining ~300 epochs regardless of the schedule's own lr decaying toward zero.

• Checking the network's internal state rules out the obvious suspects: no NaNs, weight magnitudes stay bounded, and roughly 44-48% of ReLU units are still active (not dead). The dense output layers have instead collapsed into a configuration that produces nearly-identical logits for every class regardless of input -- a symmetric, non-discriminative optimum that a tiny gradient can't easily escape.

Term refresher — representation collapse: When a network's internal representations lose the ability to distinguish between different inputs -- here, the final layer produces almost the same output regardless of what image it's shown, rather than failing via NaNs or exploding weights, which is what makes it easy to miss without directly inspecting the predicted probabilities.

Why choose this? A high starting lr under a DECAYING schedule can, in principle, combine fast early progress with eventual fine-grained stability -- that's the whole motivation for trying it here.

Why NOT this (tradeoffs)? The schedule's decaying TAIL does not protect against damage its own elevated HEAD can do early in training. If a rate is high enough to break a network in the first ~100 epochs, decaying it afterward does not necessarily provide a way back -- this is a genuinely worse outcome than just picking a fixed rate that happens to be somewhat too high.

5. Attempt 2: Safe Step Decay, and the Final Verdict

Start here (no prior knowledge assumed): Attempt 1 showed that starting HIGH is dangerous. This topic tries the safer half of the idea: start at Day 44's own known-stable rate (0.08) and only decay from there, never exceeding it.

Topic specification: step_decay_schedule(lr0=0.08, decay_factor=0.5, decay_every=100) -- identical formula to Topic 3, applied to the wide network instead, same 400-epoch budget and Day 44 dataset convention as Attempt 1.

Actual output:

config	train_acc	test_acc
Fixed lr=0.08 (Day 44 baseline)	0.995	0.537
Cosine 0.16→0.0005 (Attempt 1)	0.165	0.225
Step decay 0.08→0.01 (Attempt 2)	0.980	0.512

Actual output from this lesson's run.

Line-by-line explanation

• Honest complication #2: this is SAFER than Attempt 1 (no collapse) but still WORSE than just leaving lr fixed at 0.08 the whole time (0.980 vs 0.995 train_acc, 0.512 vs 0.537 test_acc).

• The reason is straightforward once you look at it: Day 44's lr=0.08 run was never actually OSCILLATING or unstable -- it was just slow to finish the last little bit of training. Cutting the rate in half every 100 epochs slows it down FURTHER for no corresponding stability benefit, since there was no instability to fix in the first place.

Real, honest final verdict: NEITHER schedule beat Day 44's plain fixed lr=0.08 on this network. Learning-rate schedules are a targeted fix for a SPECIFIC problem -- a rate causing real oscillation or instability, exactly what Topic 3 showed for Day 45's setup -- not a general-purpose 'train better' switch. WideTwoBlockConvNetRGB's 0.995 plateau was never an oscillation problem, so no schedule built from lr alone fixed it; reaching for a higher starting rate to speed things up instead reproduced (and worsened) exactly the instability Day 44 already found at fixed lr=0.12. Whatever is holding this specific network to 0.995 -- an initialization issue, a hard-to-escape point in its loss landscape, or simply needing more than 400 epochs -- is a different kind of problem than a schedule addresses, better suited to the adaptive per-parameter optimizers and initialization techniques later in this roadmap.

Watch: Learning Rate Schedules -- Step Decay, Cosine, Cyclical & More — covers exactly today's two schedules side by side, plus a few this lesson didn't build (cyclical schedules).

Watch: Unit 6.2 | Learning Rates and Learning Rate Schedulers | Part 5 — Sebastian Raschka's walkthrough of why and when schedules help, useful context for today's honest 'it didn't help here' result.

Why choose this? A safe, at-or-below-baseline decay schedule is close to a free trial -- worst case (as seen here) it costs a small amount of final accuracy within a fixed epoch budget, with no risk of catastrophic collapse.

Why NOT this (tradeoffs)? It is not a substitute for diagnosing WHY a network under-converges. Blindly trying a schedule without first checking whether the training curve actually shows oscillation (as Topic 3's diagnosis did) risks wasting the epoch budget on a fix for a problem that isn't there.

6. What This Maps To in PyTorch

Start here (no prior knowledge assumed): PyTorch ships both schedules built today as ready-made classes, so nothing here needs reimplementing in a real project.

Implementation:

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.08)
step_sched = torch.optim.lr_scheduler.StepLR(
    optimizer, step_size=100, gamma=0.5)   # matches step_decay_schedule
cos_sched = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=400, eta_min=0.0005)  # matches cosine_annealing_schedule

for epoch in range(epochs):
    train_one_epoch(...)
    step_sched.step()   # called once per epoch, like schedule_fn(epoch) here
```

Explanation: PyTorch's schedulers store the current lr internally and mutate optimizer.param_groups[0]['lr'] on .step() -- functionally identical to this lesson's net.lr = schedule_fn(epoch), just wrapped in a class with its own .step()/.get_last_lr() API instead of a plain function called with an epoch number.

Why choose this? PyTorch's built-in schedulers handle edge cases (warm restarts, chaining multiple schedulers, per-parameter-group rates) that would take real extra code to replicate from scratch.

Why NOT this (tradeoffs)? Understanding the plain-function version built today makes it much easier to read PyTorch's scheduler source or debug an unexpected lr value, rather than treating it as an opaque black box.

Interview Preparation — Technical Questions

Q: How would you implement cosine annealing from scratch given just the current epoch, lr0, lr_min, and total_epochs?

A: lr_min + 0.5 * (lr0 - lr_min) * (1 + cos(pi * epoch / total_epochs)) -- at epoch=0 this gives lr0 (cos(0)=1), and at epoch=total_epochs it gives lr_min (cos(pi)=-1).

Q: Why does a schedule need no changes to a network's forward() or backward() methods?

A: Because the learning rate is only ever read in ONE place -- the weight update line inside backward() (p -= self.lr * g). A schedule just changes what value self.lr holds before that line runs; forward() never touches lr at all.

Q: In this lesson's WideTwoBlockConvNetRGB experiment, why did checking for NaNs and dead ReLUs not explain the cosine-annealing failure?

A: Because the failure mode was different: weights stayed bounded and roughly 44-48% of ReLU units were still active (not the classic 'dead ReLU' signature), but the DENSE output layers had collapsed to producing nearly identical logits for every class -- a representation-collapse failure, not a numerical-instability one.

Q: If a training run shows late-training loss spikes, what would you check before reaching for a learning-rate schedule?

A: Whether the spikes are new/different from typical noise, whether they correlate with the loss basin narrowing (as Day 45 found), and whether a SMALLER fixed lr alone (no schedule) already fixes it -- a schedule is worth the added complexity specifically when the instability is time-varying (needs a big rate early, small rate late), not when a single smaller constant rate would do.

Q: Why might reducing training noise (e.g. via a decaying schedule) sometimes REDUCE test accuracy, as seen in this lesson's Topic 3?

A: If the noise was inadvertently acting as a regularizer -- preventing the network from settling too precisely into the training set's exact optimum -- removing it can let training fit the training set more exactly, at some cost to generalization. This is a real, observed, single-run effect here, not a universal law.

Interview Preparation — Theoretical Questions

Q: Why is a fixed learning rate fundamentally in tension with wanting both fast early progress and precise late-training convergence?

A: A rate large enough to make fast progress far from a minimum will tend to overshoot once close to that minimum (as Day 45 showed), while a rate small enough to converge precisely will be needlessly slow early on when large steps are safe and beneficial. A schedule resolves this by using different rates for different training phases instead of one compromise value.

Q: Is a learning-rate schedule guaranteed to improve training? What does today's lesson show about when it does and doesn't?

A: No. Today's lesson found a schedule DID help calm a training run that was genuinely oscillating (Topic 3), but provided no benefit -- and in one case made things catastrophically worse -- for a network that was slow rather than oscillating (Topics 4-5). A schedule targets a specific symptom (instability), not underperformance in general.

Q: Why might a schedule's decaying tail fail to 'undo' damage caused during its own high-lr early phase?

A: Once a network's parameters land in a degenerate region of the loss landscape (e.g. a symmetric configuration where the gradient with respect to breaking that symmetry is itself near zero), a smaller learning rate later doesn't provide a stronger signal to escape it -- it just takes smaller, still-ineffective steps within that same degenerate region.

Q: How does cosine annealing's smooth decay differ philosophically from step decay's sudden jumps, and when might that difference matter?

A: Step decay creates a discontinuity in the training dynamics at each decay point -- the network can appear to 'settle' just before a drop and then behave differently just after. Cosine annealing avoids any such discontinuity, which can matter more in longer or more sensitive training runs, though this lesson's own experiments didn't isolate that specific difference since the cosine run failed for an unrelated reason (starting lr too high).

Q: Why does this lesson insist on running BOTH a 'safe' and an 'aggressive' schedule variant rather than just one?

A: Because a single schedule attempt can't distinguish 'schedules don't help this problem' from 'this specific schedule's parameters were wrong.' Running both a conservative (never exceeds the known-safe rate) and an aggressive (starts above it) variant isolates whether the STARTING rate itself is the risk factor -- which turned out to be exactly the case here.

Key Takeaway and What's Next

Key takeaway: Learning-rate schedules -- step decay and cosine annealing, both built from scratch today -- plug into existing training code through a single mutable attribute (net.lr) with zero changes to forward() or backward(). A schedule genuinely calmed Day 45's real late-training oscillation (noise_std 0.248→0.049), though even that fix cost a little accuracy. But neither schedule fixed Day 44's non-converging wide network -- a safe version (starting at 0.08) ended up slightly worse than no schedule at all (train_acc 0.980 vs 0.995), and an aggressive version (starting at 0.16) caused a permanent, catastrophic collapse to random-guessing performance (train_acc 0.165) that its own eventual lr decay never undid. The honest lesson: a schedule is a precise tool for a specific symptom (rate-driven instability), not a general-purpose fix for a network that's merely slow or stuck.

Suggested watch order, if starting from zero

- 1. "Learning Rate Schedules -- Step Decay, Cosine, Cyclical & More" -- covers today's two schedules directly, side by side.
- 2. "Unit 6.2 | Learning Rates and Learning Rate Schedulers | Part 5" (Sebastian Raschka) -- deeper context on why and when schedules help.

Day 47 preview

- `Classic CNN architectures I` -- recreating a small LeNet-5-style network by hand on the junction dataset, moving past today's hyperparameter-tuning detour into architecture design proper.